# JAVA PROGRAM REPORT

## Bank Account Management System

| | |
|---|---|
| **Programming Language:** | Java |
| **Topic:** | Inheritance and Banking Operations |
| **Program File:** | BankManagementSystem.java |

## 1. AIM

To implement a bank account management program using Java classes, constructors, inheritance, method overriding, deposits, withdrawals, interest calculation, overdraft handling, and fund transfers.

## 2. DESCRIPTION

Account stores common account details and operations. SavingsAccount inherits Account and adds interest calculation. CurrentAccount inherits Account and overrides withdraw() to allow withdrawal within the balance plus the overdraft limit.

## 3. ALGORITHM

1. Define Account and its fields, constructor, and transaction methods.
2. Define SavingsAccount with calculateInterest().
3. Define CurrentAccount with an overdraft limit and overridden withdraw().
4. Create both account objects.
5. Display initial details and perform transactions.
6. Display updated details and final balances.

## 4. CLASS RELATIONSHIP

```
            Account
            /     \
  SavingsAccount   CurrentAccount
  interestRate     overdraftLimit
  calculateInterest()  withdraw()
```

# 5. SOURCE CODE

File name: BankManagementSystem.java

```
class Account {
    int accountNumber;
    String accountHolderName;
    double balance;
    String accountType;
    Account(int no, String name, double balance, String type) {
        accountNumber = no;
        accountHolderName = name;
        this.balance = balance;
        accountType = type;
    }
    void deposit(double amount) {
        balance = balance + amount;
        System.out.println("Deposited: " + amount);
    }
    void withdraw(double amount) {
        if (amount <= balance) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Insufficient Balance");
        }
    }
    void transfer(Account a, double amount) {
        if (amount <= balance) {
            balance = balance - amount;
            a.balance = a.balance + amount;
            System.out.println("Transferred: " + amount);
        } else {
            System.out.println("Insufficient Balance");
        }
    }
    void displayAccountDetails() {
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Account Holder: " + accountHolderName);
        System.out.println("Account Type: " + accountType);
        System.out.println("Balance: " + balance);
    }
}
class SavingsAccount extends Account {
    double interestRate;
    SavingsAccount(int no, String name, double balance, double rate) {
        super(no, name, balance, "Savings");
        interestRate = rate;
    }
    void calculateInterest() {
        double interest = balance * interestRate / 100;
        balance = balance + interest;
        System.out.println("Interest: " + interest);
    }
}
class CurrentAccount extends Account {
    double overdraftLimit;
    CurrentAccount(int no, String name, double balance, double limit) {
        super(no, name, balance, "Current");
        overdraftLimit = limit;
    }
    @Override
    void withdraw(double amount) {
        if (amount <= balance + overdraftLimit) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Overdraft Limit Exceeded");
        }
    }
}
```

# 5. SOURCE CODE (CONTINUED)

```
public class BankManagementSystem {
    public static void main(String[] args) {
        SavingsAccount s = new SavingsAccount(101, "Ravi", 10000, 5);
        CurrentAccount c = new CurrentAccount(102, "Suresh", 5000, 3000);
        System.out.println("Before Transactions:");
        s.displayAccountDetails();
        c.displayAccountDetails();
        System.out.println("\nTransactions:");
        s.deposit(2000);
        s.withdraw(1000);
        s.calculateInterest();
        c.deposit(1000);
        c.withdraw(7000);
        s.transfer(c, 2000);
        System.out.println("\nAfter Transactions:");
        s.displayAccountDetails();
        c.displayAccountDetails();
        System.out.println("\nFinal Balance:");
        System.out.println("Savings: " + s.balance);
        System.out.println("Current: " + c.balance);
    }
}
```

## 6. PROGRAM EXPLANATION

1. Account Class: Stores account number, holder name, balance, and account type, and provides common operations.

2. SavingsAccount Class: Inherits Account and calculates interest using balance × rate / 100, then adds it to the balance.

3. CurrentAccount Class: Overrides withdraw() to permit withdrawals up to balance + overdraftLimit.

4. Main Method: Creates both account types, performs transactions, transfers money, and displays final balances.

## 7. EXECUTION AND OUTPUT

Compile and run with:

```
C:\Java> javac BankManagementSystem.java
C:\Java> java BankManagementSystem
```

Output from the compiled program:

```
Before Transactions:
Account Number: 101
Account Holder: Ravi
Account Type: Savings
Balance: 10000.0
Account Number: 102
Account Holder: Suresh
Account Type: Current
Balance: 5000.0

Transactions:
Deposited: 2000.0
Withdrawn: 1000.0
Interest: 550.0
Deposited: 1000.0
Withdrawn: 7000.0
Transferred: 2000.0

After Transactions:
Account Number: 101
Account Holder: Ravi
Account Type: Savings
Balance: 9550.0
Account Number: 102
Account Holder: Suresh
Account Type: Current
Balance: 1000.0

Final Balance:
Savings: 9550.0
Current: 1000.0
```

## 8. TEST CASE SUMMARY

| Test Case | Expected Result | Actual Result | Status |
|---|---|---|---|
| Compile/run | No compilation or runtime error | Executed successfully | PASS |
| Savings transactions | Deposits, withdrawal, interest, transfer update balances | Balances update as coded | PASS |
| Current withdrawal | Withdrawal allowed within balance + overdraft limit | 7000 allowed from 6000 balance with 3000 limit | PASS |

## 9. RESULT

The supplied Bank Management System code compiled and ran successfully. It demonstrates inheritance, method overriding, deposits, withdrawals, savings interest, overdraft handling, and fund transfer.

Note: CurrentAccount.withdraw() may make the balance negative when the overdraft is used. transfer() checks only the sender's balance, not the overdraft limit.

## 10. CONCLUSION

The program shows how a common Account class can be extended by SavingsAccount and CurrentAccount to provide specialized banking behavior. The printed output follows the supplied code.